COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 13, 2026	**DATE SUBMITTED:**	Oct 15, 2026
NAME:	Mike Justin Cabangon Denzen Igloso	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 ALGORITHMIC STRATEGIES

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Fibonacci	Solves problems by dividing them into smaller subproblems
Brute Force	Linked Search	Finding the match is where it starts on the beginning of the list from left to right, all the way to the end manually. Mismatch is detected after the matching finish.
Backtracking	DFS	It tries different possible solutions and goes back if a choice does not lead to the desired result
Greedy	Selection sort	Example of coin change, It starts at the beginning of the list and finds the highest value that can be subtracted in the change until it equals to zero.
Divide-and-Conquer	Merge Sort Quick Sort	It divides the problem into smaller parts using recursive calls and solves the smaller parts and recombines them together into one solved problem

Table 12-1. Algorithmic Strategies and Examples

Screenshot

proce.cpp > getMinSteps(int)

```cpp
#include <iostream>
#include <algorithm>

const int n = 10;
int memo[n+1];

int getMinSteps(int n){
    if (n == 1) return 0;
    if (memo[n] != -1) return memo[n];
    int r = 1 + getMinSteps(n - 1);
    if (n % 2 == 0) r = std::min(r, 1 + getMinSteps(n / 2));
    if (n % 3 == 0) r = std::min(r, 1 + getMinSteps(n / 3));
    memo[n] = r;
    return r;
}

int main(){
    std::fill(memo, memo + n + 1, -1);
    std::cout << getMinSteps(n) << std::endl;
    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Emptech\IGLOS
3
```

Analysis

This program uses recursion with memoization to find the fewest steps to bring n down to 1 by trying to subtract 1, divide by 2, and divide by 3 at each number and keeping the smallest result. The memo array saves answers already solved so they aren't computed again. For n = 10, the output is 3, 10 to 9 to 3 to 1, which is better than the greedy path that takes 4 steps.

Table 12-2. Memoization Implementation

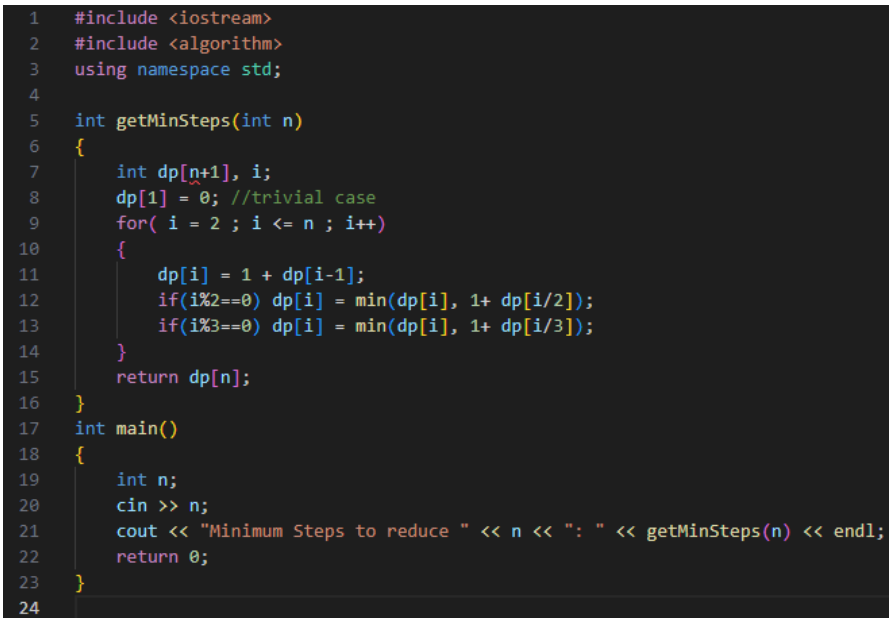
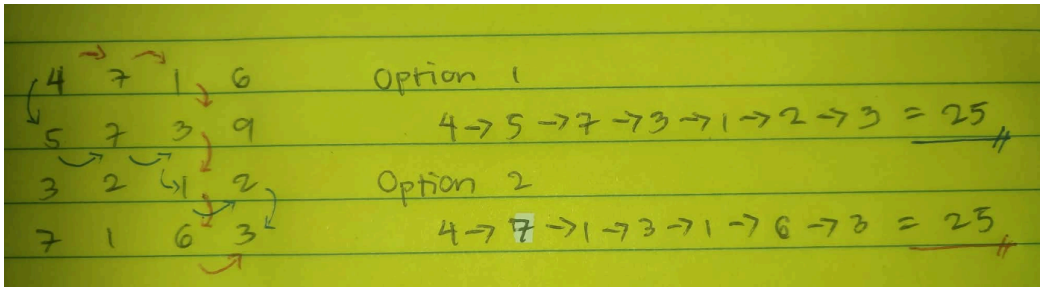
Screenshot	
Analysis	It finds the minimum steps to reduce n to 1 using subtract 1, divide by 2, or divide by 3, solved with dynamic programming. Each dp[i] takes the best of the three options, so it runs in O(n) time and space.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode	countPaths(i, j, cost):
	if i < 0 or j < 0 or cost < 0:
	return 0
	if i == 0 and j == 0:
	if mat[0][0] == cost: return 1
	else: return 0
	return countPaths(i-1, j, cost - mat[i][j]) + countPaths(i, j-1, cost - mat[i][j])

4 7 1 6 5 7 3 9 3 2 3 9 7 1 6 3	

Code

```cpp
#include <iostream>

const int M = 4;
const int N = 4;

int mat[M][N] = {
    {4, 7, 1, 6},
    {5, 7, 3, 9},
    {3, 2, 1, 2},
    {7, 1, 6, 3}};

int countPaths(int i, int j, int cost){
    if (cost < 0 || i < 0 || j < 0) return 0;
    if (i == 0 && j == 0){
        if (mat[0][0] == cost)
            return 1;
        else
            return 0;
    }
    return countPaths(i - 1, j, cost - mat[i][j]) +
           countPaths(i, j - 1, cost - mat[i][j]);
}

int main(){
    int cost = 25;
    std::cout << "Number of paths: " << countPaths(M - 1, N - 1, cost) << std::endl;
    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Emptech
Number of paths: 2
PS C:\msys64\ucrt64\bin>
```

Analysis

This program calculates the number of paths starting from the top-left cell and ending at the bottom-right cell whose sum equals a certain cost where the movement is restricted to right or downwards. This is done by going backwards from the final cell and subtracting the value of each cell while considering the cell above and the cell to the left. If the cost becomes negative or leaves the matrix then this is not counted as a valid path, and at (0, 0) the path is counted only if the remaining cost equals the value of the cell. For the 4 × 4 matrix with a cost of 25, the output is 2, which is the same result obtained manually. The algorithm takes a long time for large matrices since the same cells are solved repeatedly.

Conclusion

- During this exercise, we were taught how to solve problems using recursion by dividing them into smaller problems such as the number of ways of getting a certain cost from the cells of a matrix. The output obtained from the program was 2, which is the same output that we got when we solved it manually. We learned that recursion is inefficient since it involves solving the same cell multiple times, and hence memoization would make it more efficient.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.